

Institución Universitaria de Colombia
Facultad de Ingeniería
Ingeniería de Software

SISTEMA EXPERTO PARA DIAGNÓSTICO BÁSICO DE FALLAS EN REDES

Informe Técnico del Proyecto Integrador

Integrantes:

Juan Sebastián García Redondo

David Alexander Acosta Pineda

Ashlee Valentina Tapia Medina

Docente:

Ingrid Johanna Gómez Leguizamón

Bogotá D.C., Colombia - Julio 2026

Tabla de Contenido

Resumen Ejecutivo

Palabras Clave

Capítulo 1. Introducción

- 1.1 Contexto
- 1.2 Importancia
- 1.3 Alcance
- 1.4 Motivación

Capítulo 2. Planteamiento del Problema

- 2.1 Descripción del problema
- 2.2 Causas
- 2.3 Consecuencias
- 2.4 Justificación
- 2.5 Pregunta problema

Capítulo 3. Objetivos

- 3.1 Objetivo general
- 3.2 Objetivos específicos

Capítulo 4. Marco Conceptual

- 4.1 - 4.16 Conceptos de IA, ML, DL, Sistemas Expertos, Ontologías, LLM, etc.

Capítulo 5. Metodología

- 5.1 Metodología ABP
- 5.2 Etapas del proyecto
- 5.3 Herramientas utilizadas
- 5.4 Cronograma

Capítulo 6. Desarrollo del Proyecto

- Sesión 6: Representación del conocimiento
- Sesión 7: Lógica y reglas IF-THEN
- Sesión 8: Lógica de predicados y base de conocimiento
- Sesión 9: Sistema experto
- Sesión 10: Ontología y grafo de conocimiento
- Sesión 11: Dataset y Machine Learning
- Sesión 12: Modelo supervisado
- Sesión 13: Modelo no supervisado
- Sesión 14: Red neuronal
- Sesión 15: Deep Learning
- Sesión 16: IA Generativa
- Sesión 17: Modelo LLM
- Sesión 18: Prompt Engineering

Sesión 19: Ética y gobernanza

Capítulo 7. Arquitectura General

Capítulo 8. Resultados

8.1 Sistema experto y base de conocimiento

8.2 Ontología y grafo de conocimiento

8.3 Modelos de Machine Learning

8.4 Evidencias

Capítulo 9. Discusión

Capítulo 10. Conclusiones

Capítulo 11. Trabajo Futuro

Referencias Bibliográficas

Anexos

Lista de Figuras

Figura 1. Arquitectura del sistema experto aplicada al diagnóstico de fallas de red.

Figura 2. Grafo de conocimiento del sistema de diagnóstico de red.

Figura 3. Infografía: relación entre IA, Machine Learning y Deep Learning.

Figura 4. Arquitectura general integrada del sistema (reglas + ontología + ML + LLM).

Figura 5. Arquitectura de la red neuronal artificial (MLP) usada para el diagnóstico.

Figura 6. Método del codo para la selección del número de clústeres (K-Means).

Figura 7. Proyección PCA de los clústeres encontrados vs. el diagnóstico real.

Figura 8. Curva de aprendizaje (loss) de la red neuronal artificial.

Figura 9. Matriz de confusión de la red neuronal artificial.

Lista de Tablas

Tabla 1. Lógica proposicional vs. lógica de predicados.

Tabla 2. Reglas IF-THEN del sistema experto (15 reglas).

Tabla 3. Comparación entre bases de datos tradicionales, ontologías y grafos de conocimiento.

Tabla 4. Ciclo de vida de un proyecto de Machine Learning aplicado al proyecto.

Tabla 5. Resultados del modelo supervisado (Árbol de Decisión vs. Random Forest).

Tabla 6. Definición y uso de las métricas de evaluación.

Tabla 7. Resultados del modelo no supervisado y de la red neuronal artificial.

Tabla 8. Cronograma de actividades del proyecto (sesiones 6 a 19).

Tabla 9. Comparación general de los cuatro modelos de Machine Learning evaluados.

Resumen Ejecutivo

El diagnóstico de fallas de red en pequeñas y medianas organizaciones depende hoy, en gran medida, de la experiencia acumulada de técnicos de soporte que no siempre están disponibles de forma inmediata, lo que genera tiempos de resolución de incidentes prolongados y una alta dependencia de personal especializado. Este proyecto aborda dicho problema mediante el diseño y la implementación de un Sistema Experto para Diagnóstico Básico de Fallas en Redes, capaz de capturar el conocimiento de un técnico experto en forma de reglas lógicas y de complementarlo con técnicas de aprendizaje automático.

El proyecto integra técnicas de Inteligencia Artificial simbólica y subsimbólica: lógica de predicados y programación en Prolog para representar hechos y reglas; un motor de inferencia basado en reglas IF-THEN (15 reglas) que constituye el núcleo del sistema experto; una ontología formal en OWL y un grafo de conocimiento que organizan semánticamente los dispositivos, fallas, usuarios y acciones correctivas del dominio; y finalmente un conjunto de modelos de Machine Learning —Random Forest, K-Means y una red neuronal artificial (Perceptrón Multicapa)— entrenados sobre un dataset de 600 casos sintéticos de soporte técnico, coherentes con las reglas del sistema experto.

Como resultado, se obtuvo un sistema experto funcional en Prolog capaz de responder consultas de diagnóstico con una justificación explicable, una ontología reutilizable con 5 clases, 10 individuos y 5 relaciones, un grafo de conocimiento que revela causas raíz comunes entre fallas aparentemente distintas, y un modelo supervisado (Random Forest) que alcanzó una exactitud (Accuracy) de 0.917 y un F1-Score macro de 0.859 sobre el conjunto de prueba, superando tanto a un árbol de decisión simple como a la red neuronal entrenada bajo las mismas condiciones. Adicionalmente, se propone una arquitectura de integración con un chatbot basado en modelos de lenguaje (LLM) y técnicas de Prompt Engineering, que permitiría a un usuario no técnico describir su problema en lenguaje natural y recibir un diagnóstico junto con una explicación comprensible.

Los resultados evidencian que la combinación de un enfoque simbólico (reglas, ontologías) con un enfoque de aprendizaje automático permite construir un sistema de diagnóstico más robusto, explicable y adaptable que cualquiera de los dos enfoques por separado, sentando las bases para futuras extensiones hacia el monitoreo en tiempo real y la detección de anomalías mediante Deep Learning.

Palabras Clave

Sistema experto; Inteligencia Artificial; Diagnóstico de redes; Aprendizaje automático; Ontología.

Capítulo 1. Introducción

1.1 Contexto

Las redes de datos son la columna vertebral de la operación de cualquier organización moderna: desde el acceso a internet y el correo electrónico hasta los sistemas empresariales críticos dependen de una conectividad estable. Sin embargo, las fallas de red —caídas de enlace, errores de configuración DNS, conflictos de direccionamiento IP, saturación de ancho de banda, entre otras— son eventos frecuentes que suelen requerir la intervención de personal técnico especializado para su diagnóstico y resolución.

En este contexto, los Sistemas Expertos —una de las aplicaciones más maduras de la Inteligencia Artificial simbólica— ofrecen una alternativa viable para capturar el conocimiento de un técnico experto y ponerlo a disposición de usuarios menos especializados, las 24 horas del día, de forma consistente y explicable (Giarratano & Riley, 2005; Jackson, 1999).

1.2 Importancia

Contar con una primera línea de diagnóstico automatizada reduce el tiempo medio de resolución de incidentes (MTTR), disminuye la carga operativa sobre el personal técnico y democratiza el acceso al conocimiento experto, especialmente en organizaciones pequeñas que no cuentan con un equipo de soporte permanente. Además, la incorporación de Machine Learning permite que el sistema mejore progresivamente su capacidad de diagnóstico a partir de casos históricos, superando las limitaciones de un enfoque puramente basado en reglas estáticas.

1.3 Alcance

El proyecto abarca el diseño de una base de conocimiento en lógica de predicados (Prolog), un sistema experto con 15 reglas de diagnóstico, una ontología formal en OWL, un grafo de conocimiento, un dataset sintético de 600 casos de soporte técnico y cuatro modelos de Machine Learning (Árbol de Decisión, Random Forest, K-Means y una red neuronal artificial). Se excluyen del alcance la implementación de una interfaz gráfica de producción, la integración con hardware de red real y el despliegue en un entorno productivo, quedando estos como trabajo futuro (Capítulo 11).

1.4 Motivación

El equipo se motivó por la posibilidad de aplicar, en un mismo proyecto integrador, el espectro completo de técnicas estudiadas durante el curso —desde la lógica de predicados hasta la Inteligencia Artificial generativa— sobre un problema cotidiano y tangible como el soporte técnico de redes, en el cual cada integrante ha tenido experiencia como usuario o como soporte informal en algún momento.

Capítulo 2. Planteamiento del Problema

2.1 Descripción del problema

Cuando un usuario final experimenta una falla de conectividad, generalmente no cuenta con los conocimientos técnicos necesarios para identificar la causa raíz del problema (capa física, direccionamiento IP, resolución DNS, autenticación, etc.), lo que lo obliga a reportar el incidente y esperar la disponibilidad de un técnico. Este proceso manual es lento, poco escalable y depende completamente del conocimiento tácito de un número reducido de personas.

2.2 Causas

- Ausencia de un mecanismo de autodiagnóstico accesible para usuarios no técnicos.
- Conocimiento experto concentrado en pocas personas dentro de la organización (bás factor bajo).
- Falta de documentación estructurada y consultable de los síntomas y soluciones más comunes.
- Diversidad de posibles causas para un mismo síntoma observable (por ejemplo, "no hay internet" puede deberse a al menos cinco causas distintas).

2.3 Consecuencias

- Incremento del tiempo medio de resolución de incidentes (MTTR) y de la insatisfacción del usuario.
- Sobrecarga del personal técnico con incidentes de baja complejidad que podrían resolverse de forma autónoma.
- Pérdida de productividad de los usuarios y, en escenarios empresariales, pérdidas económicas asociadas al tiempo de inactividad.
- Dificultad para identificar patrones recurrentes de falla al no existir un registro estructurado y analizable de los incidentes.

2.4 Justificación

Un sistema experto de diagnóstico básico, complementado con modelos de Machine Learning entrenados sobre el histórico de incidentes, permite trasladar el conocimiento técnico a una herramienta disponible en todo momento, reduce la dependencia de personal especializado para los casos más comunes y genera, como subproducto, un registro estructurado de fallas que puede analizarse para la mejora continua del servicio de red.

2.5 Pregunta problema

¿En qué medida la integración de un sistema experto basado en reglas con modelos de aprendizaje automático permite automatizar y mejorar la precisión del diagnóstico básico de fallas de red respecto a un enfoque basado únicamente en reglas estáticas o únicamente en modelos de Machine Learning?

Capítulo 3. Objetivos

3.1 Objetivo general

Diseñar e implementar un sistema experto para el diagnóstico básico de fallas de red que integre representación del conocimiento mediante lógica de predicados, un motor de inferencia basado en reglas, una ontología y un grafo de conocimiento, complementado con modelos de aprendizaje automático supervisado y no supervisado, con el fin de mejorar la precisión, la explicabilidad y la accesibilidad del proceso de diagnóstico.

3.2 Objetivos específicos

1. Representar el conocimiento experto sobre fallas de red mediante hechos y reglas en lógica de predicados, implementados en Prolog.
2. Diseñar un sistema experto con al menos 15 reglas IF-THEN capaces de inferir un diagnóstico y una acción correctiva a partir de los síntomas reportados.
3. Construir una ontología formal en OWL y un grafo de conocimiento que representen semánticamente los dispositivos, fallas, usuarios y acciones del dominio.
4. Construir un dataset representativo de casos de soporte técnico y entrenar modelos de aprendizaje supervisado (Árbol de Decisión, Random Forest) y no supervisado (K-Means) para el diagnóstico de fallas.
5. Entrenar y evaluar una red neuronal artificial y compararla con los modelos de Machine Learning clásico en términos de Accuracy, Precision, Recall y F1-Score.
6. Proponer una arquitectura de integración de todos los componentes anteriores con una interfaz conversacional basada en modelos de lenguaje (LLM) y técnicas de Prompt Engineering.
7. Analizar las implicaciones éticas y de gobernanza asociadas al uso de Inteligencia Artificial en el diagnóstico de infraestructura de red.

Capítulo 4. Marco Conceptual

4.1 Inteligencia Artificial

La Inteligencia Artificial (IA) se define como el campo de la informática dedicado a la construcción de sistemas capaces de realizar tareas que, si fueran ejecutadas por un humano, requerirían inteligencia: percibir, razonar, aprender, planificar y actuar (Russell & Norvig, 2021). Dentro de este campo conviven enfoques simbólicos —basados en lógica y reglas, como el sistema experto de este proyecto— y enfoques subsimbólicos o conexionistas, basados en el aprendizaje a partir de datos, como las redes neuronales.

4.2 IA Débil

La IA débil (o estrecha) corresponde a sistemas diseñados para resolver una tarea específica sin poseer una comprensión general del mundo, a diferencia de la IA fuerte o general, que sigue siendo un objetivo teórico no alcanzado (Russell & Norvig, 2021). El sistema experto y los modelos de Machine Learning desarrollados en este proyecto son ejemplos de IA débil: cada componente resuelve una tarea acotada (inferir un diagnóstico, clasificar un caso, agrupar síntomas similares) sin generalizar a otros dominios.

4.3 Agentes Inteligentes

Un agente inteligente es toda entidad capaz de percibir su entorno mediante sensores y actuar sobre él mediante actuadores, persiguiendo un objetivo definido (Russell & Norvig, 2021). El sistema experto propuesto puede modelarse como un agente basado en reglas: percibe el entorno a través de los síntomas reportados por el usuario (sensores lógicos), razona mediante su motor de inferencia y actúa recomendando un diagnóstico y una acción correctiva (actuador informativo).

4.4 Representación del Conocimiento

La representación del conocimiento es el conjunto de técnicas formales utilizadas para codificar información sobre el mundo de manera que un sistema computacional pueda utilizarla para razonar (Sowa, 2000). En este proyecto se emplean tres formas complementarias de representación: reglas de producción IF-THEN, lógica de predicados en Prolog y una ontología formal en OWL, cada una con distinto nivel de expresividad y capacidad de inferencia.

4.5 Lógica de Predicados

A diferencia de la lógica proposicional, que solo permite evaluar la verdad de proposiciones completas, la lógica de predicados permite expresar propiedades y relaciones entre objetos mediante predicados, variables y cuantificadores universales ($\forall$) y existenciales ($\exists$) (Russell & Norvig, 2021). Esta mayor expresividad es la que permite escribir reglas generales como `diagnostico(X, falla_hardware_fisico) :- cable_desconectado(X)`, aplicable a cualquier dispositivo X, y no solo a un caso particular.

4.6 Sistemas Expertos

Un sistema experto es un programa que emula la capacidad de toma de decisiones de un experto humano en un dominio específico, mediante una base de conocimiento (hechos y reglas) y un motor de inferencia que aplica dichas reglas para llegar a conclusiones (Giarratano & Riley, 2005). Los sistemas expertos se caracterizan por su capacidad de explicar el razonamiento que sustenta cada conclusión, una propiedad que los modelos de Machine Learning de caja negra no siempre ofrecen (Jackson, 1999).

4.7 Ontologías

En el contexto de la IA, una ontología es una especificación formal y explícita de una conceptualización compartida: define las clases, propiedades y relaciones de un dominio de manera que puedan ser interpretadas de forma consistente por distintos sistemas (Gruber, 1993). El proceso de construcción de una ontología incluye identificar las clases del dominio, sus relaciones (propiedades de objeto) y sus instancias (individuos), tal como se describe en la metodología de Noy y McGuinness (2001), seguida en el Capítulo 6 de este informe.

4.8 Grafos de Conocimiento

Un grafo de conocimiento es una estructura de datos que representa entidades del mundo real como nodos y sus relaciones como aristas etiquetadas, habitualmente construida sobre el vocabulario formal de una ontología (Hogan et al., 2021). Los grafos de conocimiento permiten realizar consultas de recorrido (traversal) que revelan relaciones no evidentes en una estructura tabular, como identificar que múltiples fallas reportadas por distintos usuarios comparten un mismo dispositivo como causa raíz.

4.9 Machine Learning

El aprendizaje automático (Machine Learning) es el subcampo de la IA que estudia algoritmos capaces de mejorar su desempeño en una tarea a partir de la experiencia, es decir, a partir de datos, sin ser programados explícitamente regla por regla (Mitchell, 1997). A diferencia del

sistema experto —cuyas reglas son escritas manualmente por un experto—, los modelos de Machine Learning infieren automáticamente los patrones que relacionan los síntomas con el diagnóstico a partir de casos históricos (Géron, 2022).

4.10 Aprendizaje Supervisado

El aprendizaje supervisado utiliza un conjunto de datos etiquetado, en el que cada observación cuenta con una variable objetivo conocida, para entrenar un modelo capaz de predecir dicha variable en observaciones nuevas (Bishop, 2006). En este proyecto se emplean dos algoritmos supervisados: un Árbol de Decisión y un Random Forest (Breiman, 2001), entrenados para predecir la variable diagnóstico a partir de los síntomas de red observados.

4.11 Aprendizaje No Supervisado

El aprendizaje no supervisado busca descubrir estructura o patrones en los datos sin contar con etiquetas previas (Bishop, 2006). En este proyecto se utiliza el algoritmo K-Means (MacQueen, 1967) para agrupar los casos de soporte técnico según la similitud de sus síntomas, con el fin de descubrir posibles tipos de falla no contemplados explícitamente en las reglas del sistema experto.

4.12 Redes Neuronales

Una red neuronal artificial es un modelo computacional inspirado en la estructura del cerebro biológico, compuesto por unidades (neuronas) organizadas en capas que se conectan mediante pesos ajustables durante el entrenamiento (Rosenblatt, 1958; Goodfellow et al., 2016). En este proyecto se implementó un Perceptrón Multicapa (MLP) con dos capas ocultas para clasificar el diagnóstico de red, comparando su desempeño con los modelos de Machine Learning clásico.

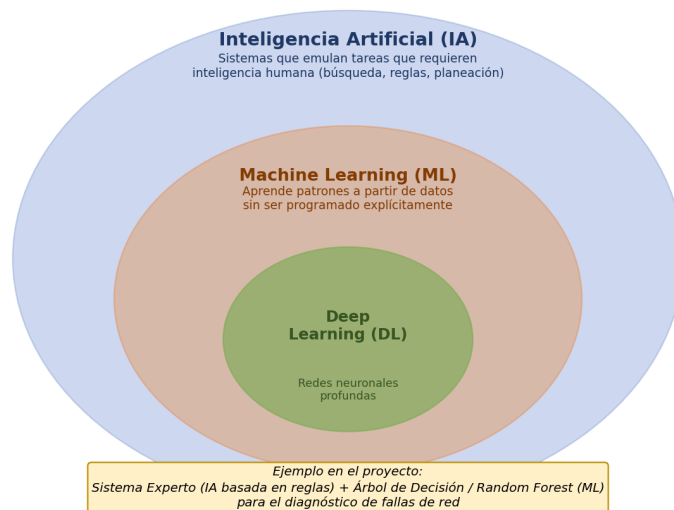


Figura 3. Infografía: relación entre IA, Machine Learning y Deep Learning.

4.13 Deep Learning

El aprendizaje profundo (Deep Learning) es un subconjunto del Machine Learning que utiliza redes neuronales con múltiples capas ocultas para aprender representaciones jerárquicas directamente de datos no estructurados como imágenes, audio o texto (LeCun, Bengio & Hinton, 2015; Goodfellow et al., 2016). Aunque el presente proyecto no requiere Deep Learning —al tratarse de datos tabulares de baja dimensionalidad—, se identifica como línea de trabajo futuro su aplicación a series temporales de tráfico de red mediante redes recurrentes (LSTM) para la detección de anomalías (ver Capítulo 11).

4.14 IA Generativa

La Inteligencia Artificial generativa comprende los modelos capaces de generar contenido nuevo —texto, imágenes, código— a partir de patrones aprendidos de grandes volúmenes de datos, siendo los modelos de lenguaje de gran escala (LLM) su exponente más relevante en el procesamiento de texto (Brown et al., 2020). En este proyecto se propone el uso de IA generativa como interfaz conversacional del sistema experto, capaz de traducir el lenguaje natural del usuario en hechos estructurados y de explicar el diagnóstico en un lenguaje comprensible.

4.15 Modelos LLM

Los Modelos de Lenguaje de Gran Escala (Large Language Models, LLM) son redes neuronales basadas en la arquitectura Transformer (Vaswani et al., 2017), entrenadas sobre

grandes corpus de texto para predecir la siguiente palabra en una secuencia, lo que les permite generalizar a tareas de comprensión y generación de lenguaje natural sin haber sido entrenadas específicamente para ellas (Brown et al., 2020; OpenAI, 2023). En la arquitectura propuesta en el Capítulo 7, un LLM cumpliría el rol de interfaz entre el usuario y el motor de inferencia del sistema experto.

4.16 Prompt Engineering

El Prompt Engineering es la disciplina que estudia el diseño de instrucciones (prompts) que maximizan la calidad y precisión de las respuestas generadas por un LLM, mediante técnicas como el uso de ejemplos (few-shot prompting), la definición de un rol o persona, y la especificación explícita del formato de salida esperado (White et al., 2023). En el Capítulo 6 (Sesión 18) se presentan los prompts diseñados para el asistente conversacional del sistema experto.

Capítulo 5. Metodología

5.1 Metodología ABP (Aprendizaje Basado en Proyectos)

El proyecto se desarrolló bajo la metodología de Aprendizaje Basado en Proyectos (ABP), en la cual los conceptos teóricos de cada sesión del curso se aplican de forma incremental sobre un mismo proyecto integrador. Esta metodología favorece la comprensión profunda de cada técnica de Inteligencia Artificial al exigir su aplicación inmediata sobre un problema real y contextualizado, en lugar de abordarla de forma aislada mediante ejercicios genéricos.

5.2 Etapas del proyecto

- Etapa 1 — Fundamentos de representación del conocimiento (sesiones 6 y 7): modelado de hechos y reglas IF-THEN sobre el dominio de fallas de red.
- Etapa 2 — Lógica formal y sistemas expertos (sesiones 8 y 9): formalización en lógica de predicados, implementación en Prolog y diseño del sistema experto de 15 reglas.
- Etapa 3 — Representación semántica (sesión 10): construcción de la ontología en Protégé (OWL) y del grafo de conocimiento.
- Etapa 4 — Machine Learning clásico (sesiones 11 a 13): construcción del dataset, y entrenamiento de modelos supervisados y no supervisados.
- Etapa 5 — Redes neuronales y Deep Learning (sesiones 14 y 15): entrenamiento de una red neuronal artificial y análisis conceptual de aplicabilidad de Deep Learning.
- Etapa 6 — IA generativa (sesiones 16 a 18): diseño conceptual de un asistente conversacional basado en LLM y de los prompts asociados.
- Etapa 7 — Ética y cierre (sesión 19 y capítulos finales): análisis ético, integración de componentes, resultados y conclusiones.

5.3 Herramientas utilizadas

Herramienta	Uso en el proyecto
SWI-Prolog	Implementación de la base de conocimiento en lógica de predicados (hechos y reglas).
Protégé	Construcción y edición de la ontología OWL del dominio.
Python 3 / Google Colab	Construcción del dataset y entrenamiento de los modelos de Machine Learning.
Scikit-Learn	Implementación de Árbol de Decisión, Random Forest, K-Means y MLPClassifier (Pedregosa et al., 2011).
Graphviz	Generación del grafo de conocimiento y los diagramas de arquitectura.
Matplotlib	Visualización de métricas, curvas de aprendizaje y matrices de confusión.

Microsoft Word / PDF	Elaboración y entrega del presente informe técnico.
----------------------	---

5.4 Cronograma

Sesión	Actividad principal	Semana
6	Representación del conocimiento (hechos y reglas iniciales)	1
7	Reglas IF-THEN del dominio de fallas de red	1
8	Lógica de predicados y base de conocimiento en Prolog	2
9	Diseño del sistema experto (15 reglas) y su arquitectura	2
10	Ontología en Protégé y grafo de conocimiento	3
11	Construcción del dataset y fundamentos de Machine Learning	3
12	Modelo supervisado (Árbol de Decisión y Random Forest)	4
13	Modelo no supervisado (K-Means)	4
14	Red neuronal artificial (MLP)	5
15	Análisis conceptual de Deep Learning	5
16	IA generativa y arquitectura del asistente conversacional	6
17	Modelos LLM aplicados al proyecto	6
18	Prompt Engineering para el asistente del sistema experto	7
19	Ética y gobernanza de IA	7
—	Integración final, redacción del informe y sustentación	8

Capítulo 6. Desarrollo del Proyecto

Este capítulo documenta, sesión por sesión, el trabajo realizado sobre el proyecto integrador, siguiendo la secuencia temática del curso desde la representación básica del conocimiento hasta la ética de la Inteligencia Artificial.

Sesión 6: Representación del conocimiento

Se inició el proyecto identificando los elementos del dominio que debían representarse: dispositivos de red (router, switch, servidor, equipos de usuario), síntomas observables (estado de LEDs, resultado de comandos ping, acceso a páginas web) y posibles fallas. Se optó por una representación basada en hechos atómicos —afirmaciones simples sobre el estado del mundo— como unidad mínima de conocimiento, por ser la forma más natural de capturar observaciones discretas de un técnico de soporte.

Ejemplos de hechos identificados en esta sesión: router_principal tiene el LED de enlace apagado; el equipo pc_finanzas tiene el cable de red desconectado; el usuario de marketing reporta que no puede abrir páginas web. Estos hechos, aunque expresados aquí en lenguaje natural, fueron la base para su posterior formalización en reglas (sesión 7) y en lógica de predicados (sesión 8).

Sesión 7: Lógica y reglas IF-THEN

A partir de los hechos identificados en la sesión anterior, se formularon las primeras reglas de producción con la estructura IF (condición) THEN (conclusión), el formalismo clásico de los sistemas basados en reglas (Giaratano & Riley, 2005). Por ejemplo: SI el cable de red está desconectado ENTONCES el diagnóstico es falla de hardware físico. Esta sesión permitió validar que el conocimiento del dominio podía expresarse de forma completa mediante combinaciones de condiciones (AND) sobre los hechos observables, sentando las bases de las 15 reglas definitivas del sistema experto (sesión 9).

Se identificaron además los primeros conflictos de reglas (por ejemplo, un mismo síntoma —"sin acceso a internet"— podía derivarse de varias causas distintas), lo cual motivó la necesidad de reglas más específicas y ordenadas por prioridad, resuelta posteriormente mediante el uso de condiciones más granulares (variables como ip_asignada, ping_router y ping_dns_externo en conjunto, en lugar de un único síntoma genérico).

Sesión 8: Lógica de predicados y base de conocimiento

Se formalizó el conocimiento del dominio en lógica de predicados y se implementó en Prolog una base de conocimiento con 10 hechos y 5 reglas de diagnóstico (Bratko, 2012; Clocksin & Mellish, 2003). A diferencia de las reglas informales de la sesión 7, en Prolog las reglas admiten variables lógicas (por ejemplo, X en `diagnostico(X, falla_hardware_fisico)`):-
`cable_desconectado(X)`), lo que permite que una sola regla se aplique a cualquier dispositivo de la red, no solo a un caso particular.

```
% ===== HECHOS =====
ping_falla(router_principal).
puerto_caido(switch_piso1).
ip_duplicada(pc_contabilidad).
dns_incorrecto(pc_marketing).
cable_desconectado(pc_finanzas).
usuario_reporta(pc_finanzas, sin_internet_total).

% ===== REGLA =====
diagnostico(D, falla_hardware_fisico) :- cable_desconectado(D).
diagnostico(D, aislamiento_red_por_router) :-
    usuario_reporta(D, sin_internet_total), ping_falla(router_principal).

% ===== CONSULTA =====
?- diagnostico(pc_finanzas, Falla).
% Falla = aislamiento_red_por_router
```

El código completo (10 hechos, 5 reglas y 5 consultas de ejemplo) se encuentra en el Anexo A (archivo `diagnostico_red.pl`). Se comprobó, mediante las consultas de ejemplo, que el motor de inferencia de Prolog (basado en resolución SLD y backward chaining) era capaz de derivar correctamente el diagnóstico esperado para cada combinación de hechos.

Sesión 9: Sistema experto

Con base en la lógica formalizada en Prolog, se diseñó la arquitectura completa del sistema experto (Figura 1), compuesta por seis módulos: interfaz de usuario, memoria de trabajo, motor de inferencia, base de conocimiento, módulo de adquisición de conocimiento y módulo de explicación (Giarratano & Riley, 2005).

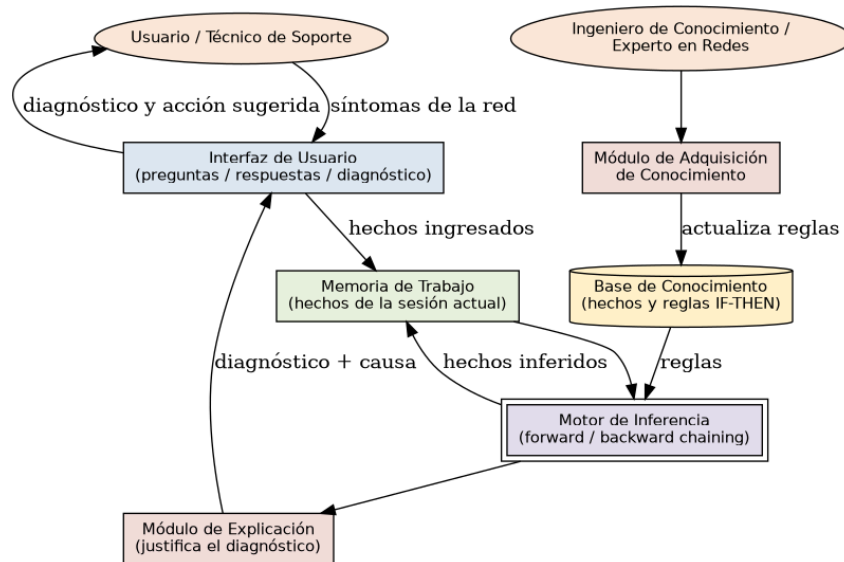


Figura 1. Arquitectura del sistema experto aplicada al diagnóstico de fallas de red.

Se amplió la base de reglas de 5 a 15 reglas IF-THEN (Tabla 2), cubriendo los escenarios más comunes de fallas domésticas y empresariales: fallas físicas de cableado, problemas de DHCP/IP, fallas de DNS, saturación de ancho de banda, bloqueos por firewall o portal cautivo, fallas de autenticación WiFi, entre otras. Cada regla incluye no solo el diagnóstico, sino también una acción correctiva sugerida, lo que constituye la salida final del módulo de explicación hacia el usuario.

#	Condición (IF)	Diagnóstico (THEN)	Acción sugerida
1	tipo_conexion=ethernet AND led_enlace_router=apagado	Falla de enlace físico (Capa 1)	Verificar el cable UTP en ambos extremos.
2	led_enlace_router=rojo	Pérdida de señal WAN/Fibra	Reiniciar el router; reportar al ISP si persiste.
3	ip_asignada=autoconfigurada_apipa AND ping_router=fallido	Fallo de servidor DHCP	Verificar el servicio DHCP / renovar IP.
4	ip_asignada=valida_dhcp AND ping_router=fallido	Aislamiento por firewall	Desactivar temporalmente el firewall.
5	ping_router=exitoso AND ping_dns_externo=fallido AND acceso_web=nulo	Caída total del enlace WAN	Validar el estado de la línea con el ISP.
6	ping_dns_externo=exitoso AND resolucion_dns=falla	Fallo crítico de DNS	Cambiar a DNS públicos (8.8.8.8 / 1.1.1.1).

7	acceso_web=parcial AND resolucion_dns=funciona	Saturación de ancho de banda	Ejecutar test de velocidad.
8	tipo_conexion=wifi AND autenticacion_usuario=rechazada	Fallo de credenciales WiFi	Reingresar la clave de red.
9	acceso_web=parcial AND ping_dns_externo=fallido	Bloqueo por portal cautivo	Abrir el navegador para forzar redirección.
10	ip_asignada=ninguna AND tipo_conexion=ethernet	Adaptador de red deshabilitado	Habilitar el adaptador / reinstalar drivers.
11	acceso_web=parcial AND led_enlace_router=parpadeante	Pérdida intermitente de paquetes	Verificar atenuación y microfiltros.
12	resolucion_dns=funciona AND acceso_web=nulo	Bloqueo de puertos / proxy mal configurado	Revisar configuración de proxy.
13	tipo_conexion=wifi AND ping_router=fallido AND led_enlace_router=verde	Saturación de canales WiFi	Cambiar el canal de la banda.
14	ip_asignada=valida_dhcp AND acceso_web=nulo	Restricción por control parental / ACL	Revisar filtrado MAC del router.
15	led_enlace_router=apagado AND tipo_conexion=wifi	Router apagado o sin energía	Verificar alimentación eléctrica.

Sesión 10: Ontología y grafo de conocimiento

Se formalizó el vocabulario del dominio mediante una ontología construida en Protégé y exportada en formato OWL (Anexo B), siguiendo la metodología de Noy y McGuinness (2001): identificación de clases (Dispositivo, Usuario, Falla, Regla_Diagnostico, Accion_Correctiva), definición de relaciones de tipo Object Property (presentaFalla, requiereAccion, reportadaPor, conectadoA, diagnosticadaPor) y creación de 10 individuos representativos del dominio.

Sobre esta misma ontología se construyó un grafo de conocimiento (Figura 2) que materializa las relaciones entre los dispositivos reales de la red simulada, permitiendo identificar visualmente que varias fallas reportadas por distintos usuarios (marketing, finanzas, contabilidad) convergen en un mismo punto de falla: el router_principal, información que no resulta evidente al examinar los incidentes de forma aislada en una tabla.

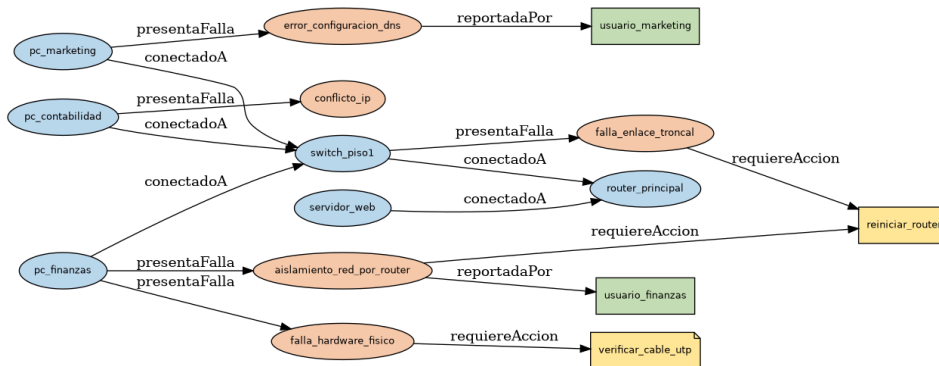


Figura 2. Grafo de conocimiento del sistema de diagnóstico de red.

Sesión 11: Dataset y Machine Learning

Se construyó un dataset sintético de 600 casos de soporte técnico (Anexo D, `dataset_diagnostico_red.csv`), generado de forma coherente con las 15 reglas del sistema experto, para poder entrenar y evaluar modelos de Machine Learning sobre el mismo dominio. El dataset cuenta con 8 variables de entrada categóricas (`tipo_conexion`, `led_enlace_router`, `ip_asignada`, `ping_router`, `ping_dns_externo`, `resolucion_dns`, `acceso_web`, `autenticacion_usuario`) y una variable objetivo diagnostico con 9 clases posibles. Se introdujo deliberadamente un 5% de ruido en las etiquetas para simular la ambigüedad presente en los diagnósticos reales.

Esta sesión permitió comparar de forma práctica el paradigma simbólico (reglas fijas, escritas por un experto) con el paradigma de aprendizaje automático (patrones inferidos automáticamente de los datos), discutido conceptualmente en el Capítulo 4.

Sesión 12: Modelo supervisado

Se entrenaron dos modelos de clasificación supervisada sobre el dataset de la sesión anterior: un Árbol de Decisión y un Random Forest de 200 árboles (Breiman, 2001), implementados en Scikit-Learn (Pedregosa et al., 2011) sobre Google Colab (Anexo E, `modelo_supervisado_diagnostico_red.ipynb`). Los resultados se presentan y analizan en detalle en el Capítulo 8 (Resultados); el Random Forest obtuvo el mejor desempeño (Accuracy = 0.917, F1-Score macro = 0.859), confirmando la robustez de los modelos de ensamble frente a un único árbol sobre datos categóricos con clases desbalanceadas.

Sesión 13: Modelo no supervisado

Se exploró la aplicación de K-Means (MacQueen, 1967) sobre el mismo conjunto de variables de entrada, sin utilizar la etiqueta diagnóstico, con el fin de evaluar si los síntomas de red se agrupan naturalmente en clústeres coherentes con los diagnósticos reales. Se utilizó el método del codo para seleccionar el número de clústeres (Figura 6) y se fijó $k=9$ para comparar directamente con las 9 categorías reales de diagnóstico.

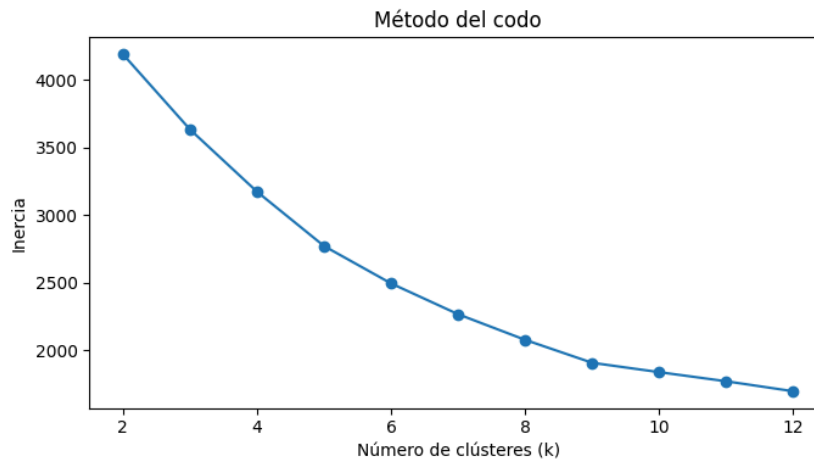


Figura 6. Método del codo para la selección del número de clústeres (K-Means).

El agrupamiento obtenido alcanzó un Silhouette Score de 0.277 y un Adjusted Rand Index de 0.184 respecto al diagnóstico real (Anexo F, modelo_no_supervisado_red_neuronal.ipynb). Estos valores, moderados, indican que si bien existe cierta estructura natural en los síntomas que se aproxima a las categorías de diagnóstico, el agrupamiento no supervisado por sí solo no reemplaza el conocimiento experto codificado en las reglas; su mayor utilidad radica en la posibilidad de identificar subgrupos de casos atípicos que podrían corresponder a fallas nuevas, no contempladas todavía en el sistema de reglas.

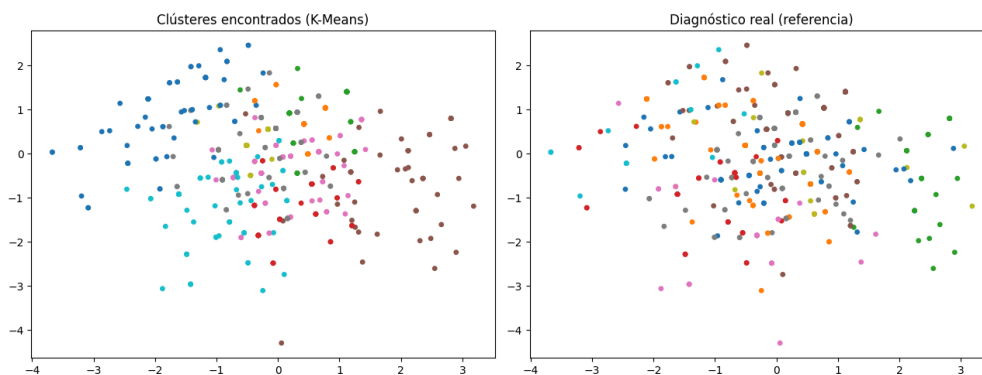


Figura 7. Proyección PCA de los clústeres encontrados vs. el diagnóstico real.

Sesión 14: Red neuronal

Se diseñó y entrenó un Perceptrón Multicapa (MLP) con dos capas ocultas (6 y 4 neuronas respectivamente, Figura 5) para predecir la variable diagnóstico, utilizando la misma división de entrenamiento/prueba que el modelo supervisado de la sesión 12, con el fin de realizar una comparación justa entre ambos enfoques.

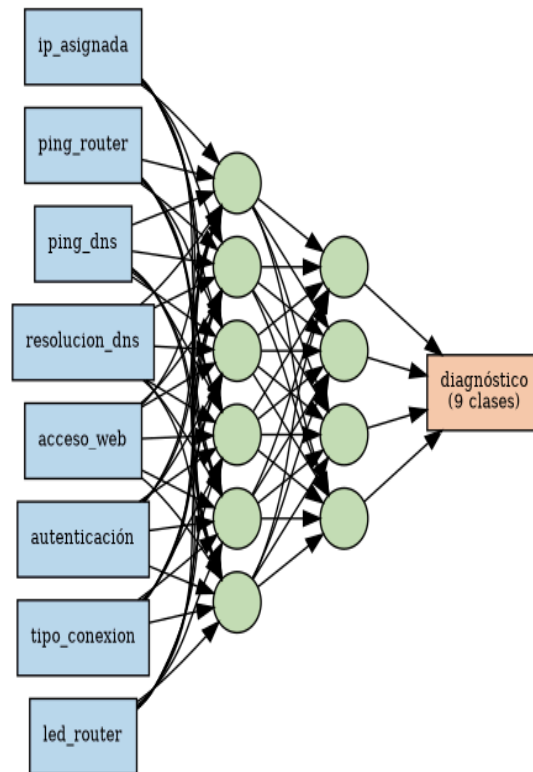


Figura 5. Arquitectura de la red neuronal artificial (MLP) usada para el diagnóstico.

La red neuronal alcanzó un Accuracy de 0.725 y un F1-Score macro de 0.499 (Tabla 7), un desempeño inferior al del Random Forest. Este resultado es consistente con la literatura especializada, que señala que los modelos de árboles y ensambles suelen igualar o superar a las redes neuronales en conjuntos de datos tabulares de tamaño pequeño o moderado, mientras que las redes neuronales muestran ventajas claras en datos de alta dimensionalidad y gran volumen (Géron, 2022).

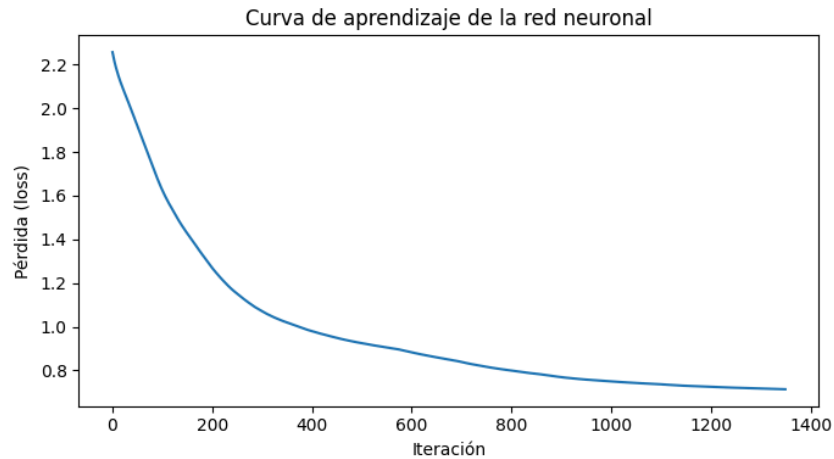


Figura 8. Curva de aprendizaje (loss) de la red neuronal artificial.

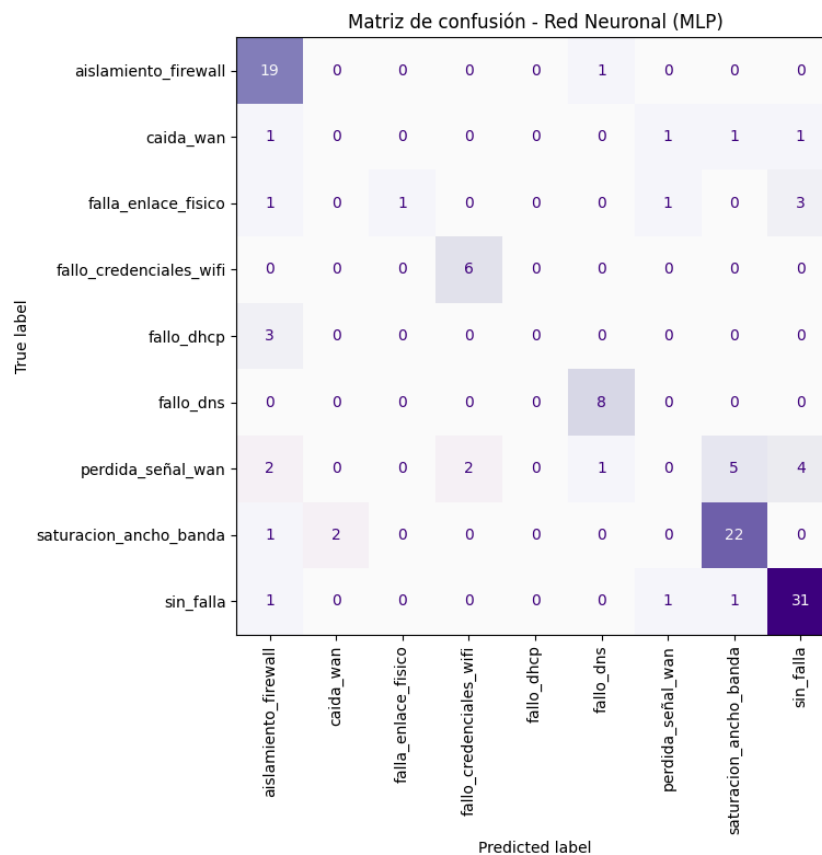


Figura 9. Matriz de confusión de la red neuronal artificial.

Sesión 15: Deep Learning

Se analizó conceptualmente la pertinencia de aplicar Deep Learning al proyecto. Dado que el dataset actual es tabular, de baja dimensionalidad (8 variables categóricas) y de tamaño moderado (600 registros), un modelo de aprendizaje profundo no aporta una ventaja

significativa frente al Random Forest, y además requeriría un volumen de datos considerablemente mayor para evitar el sobreajuste (Goodfellow et al., 2016).

No obstante, se identificó una línea de aplicación de alto valor para el proyecto: el análisis de series temporales de tráfico de red (latencia, pérdida de paquetes, throughput) mediante redes neuronales recurrentes (LSTM) o redes convolucionales 1D, capaces de aprender patrones temporales asociados a la degradación progresiva del enlace antes de una falla total, habilitando un diagnóstico predictivo en lugar de puramente reactivo (LeCun et al., 2015). Esta línea se retoma como trabajo futuro en el Capítulo 11.

Sesión 16: IA Generativa

Se diseñó, a nivel conceptual y arquitectónico, la incorporación de un componente de IA generativa como interfaz conversacional del sistema experto (Figura 4). En lugar de exigir al usuario que conozca y reporte variables técnicas específicas (`led_enlace_router`, `ping_dns_externo`, etc.), un modelo de lenguaje permitiría al usuario describir su problema en lenguaje natural ("desde ayer no me carga ninguna página, pero el WhatsApp del celular sí funciona") y extraería de dicha descripción los hechos estructurados que el motor de inferencia necesita para aplicar las reglas del sistema experto.

De igual forma, el módulo de explicación del sistema experto —que hoy generaría una salida técnica como `diagnostico(pc_marketing, error_configuracion_dns)`— podría apoyarse en el LLM para redactar una explicación clara y empática dirigida a un usuario no técnico, cerrando el ciclo de interacción en lenguaje natural.

Sesión 17: Modelo LLM

Se estudió la arquitectura Transformer (Vaswani et al., 2017), base de los modelos de lenguaje de gran escala (LLM) como GPT (OpenAI, 2023), y su mecanismo de atención, que permite al modelo ponderar la relevancia de cada palabra del contexto al generar cada nueva palabra de la respuesta. Se concluyó que, para el propósito de este proyecto, no es necesario entrenar un LLM desde cero: basta con integrar un modelo preentrenado mediante su API (como se describe en la Figura 4, módulo "Interfaz conversacional"), aprovechando sus capacidades generales de comprensión de lenguaje natural y aplicando Prompt Engineering (sesión 18) para especializarlo en el dominio de diagnóstico de redes, sin necesidad de reentrenar sus parámetros (fine-tuning).

Sesión 18: Prompt Engineering

Se diseñaron los prompts que especializarían al LLM en el rol de asistente de triage de red, aplicando las técnicas de definición de rol (persona), few-shot prompting y especificación de formato de salida estructurado (White et al., 2023), de manera que la respuesta del modelo pueda convertirse automáticamente en hechos Prolog. A continuación se muestra un ejemplo simplificado del prompt de sistema propuesto:

Rol: Eres un asistente de soporte técnico de primer nivel, especializado en fallas de red domésticas y de oficina.

Tarea: A partir de la descripción en lenguaje natural del usuario, identifica los valores de las siguientes variables y responde ÚNICAMENTE en formato JSON (sin texto adicional):

tipo_conexion, led_enlace_router, ip_asignada, ping_router, ping_dns_externo, resolucion_dns, acceso_web, autenticacion_usuario. Si un valor no puede inferirse con certeza, usa "desconocido".

Ejemplo:

Usuario: "No me carga ninguna página desde ayer, pero el WhatsApp del celular sí funciona con datos móviles."

Respuesta: {"tipo_conexion": "wifi", "acceso_web": "nulo", "led_enlace_router": "desconocido", ...}

Un segundo prompt, aplicado después de que el motor de inferencia entrega el diagnóstico técnico, se encargaría de traducirlo a lenguaje natural: "Actúa como un técnico de soporte amable. Explica en un máximo de 3 frases, sin tecnicismos, el siguiente diagnóstico y su acción sugerida: {diagnostico}, {accion_sugerida}." Esta separación en dos prompts especializados (extracción de hechos y explicación de resultados) sigue la buena práctica de dividir tareas complejas en subtarear más simples y verificables (White et al., 2023).

Sesión 19: Ética y gobernanza

Se analizaron las implicaciones éticas del sistema propuesto a la luz de los principios de la Recomendación sobre la Ética de la Inteligencia Artificial de la UNESCO (2021) y del marco de cinco principios de Floridi y Cowls (2019): beneficencia, no maleficencia, autonomía, justicia y explicabilidad.

- Privacidad y protección de datos: la información de diagnóstico de red puede revelar patrones de uso o infraestructura sensible de una organización; su tratamiento debe ajustarse a la normativa de protección de datos personales vigente en Colombia (Ley 1581 de 2012), incluso cuando los datos utilizados en este proyecto son sintéticos.
- Explicabilidad: se priorizó deliberadamente un sistema experto basado en reglas —inherentemente explicable— como primera línea de diagnóstico, reservando los modelos de Machine Learning menos transparentes (Random Forest, red neuronal) como mecanismo complementario y no como única fuente de decisión.

- Sesgo y equidad: al ser un dataset sintético construido a partir de reglas conocidas, el riesgo de sesgo discriminatorio hacia personas es bajo; sin embargo, en una implementación real con datos históricos de incidentes, deberá auditarse que el modelo no penalice sistemáticamente a ciertos tipos de usuario o ubicación por sesgos presentes en los datos de entrenamiento.
- Supervisión humana: se recomienda que el sistema opere como apoyo a la decisión (decision support) y no de forma completamente autónoma para diagnósticos críticos, manteniendo la posibilidad de escalar a un técnico humano cuando el sistema reporte baja confianza.
- Transparencia con el usuario: el usuario debe ser informado de que interactúa con un sistema automatizado (incluyendo el componente de IA generativa) y debe tener la posibilidad de solicitar la intervención de una persona.

Capítulo 7. Arquitectura General

La Figura 4 presenta la arquitectura general propuesta, que integra los seis componentes desarrollados a lo largo del proyecto: la interfaz conversacional basada en LLM (sesiones 16-18), el módulo de extracción de hechos, el motor de inferencia en Prolog (sesión 8) que consulta la base de reglas del sistema experto (sesión 9), la ontología y el grafo de conocimiento (sesión 10) que aportan el vocabulario semántico compartido, y los modelos de Machine Learning (sesiones 12-14) entrenados sobre el dataset histórico (sesión 11).

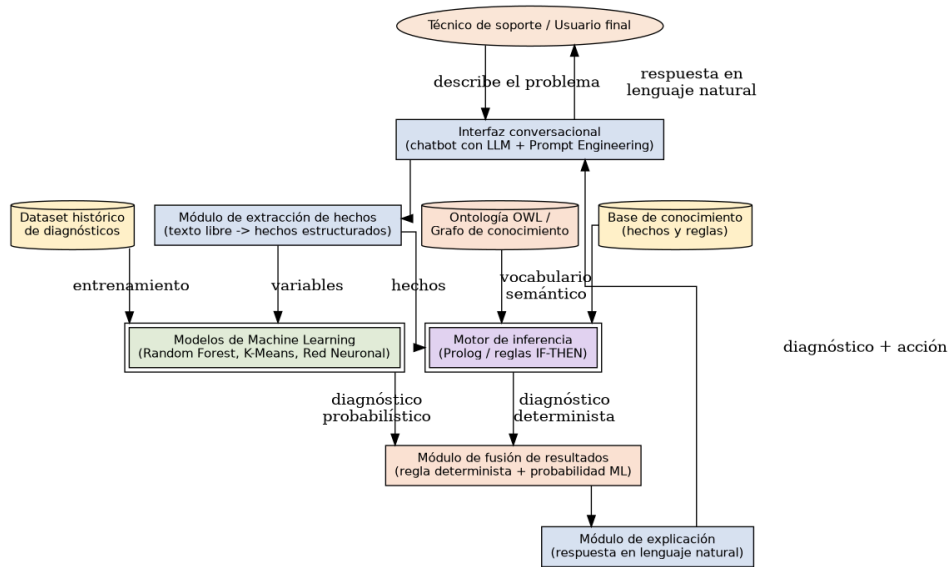


Figura 4. Arquitectura general integrada del sistema (reglas + ontología + ML + LLM).

El flujo de razonamiento propuesto sigue una estrategia híbrida en dos etapas: primero se intenta un diagnóstico determinista mediante el motor de reglas (Prolog); si ninguna regla cubre con certeza la combinación de síntomas reportados, el módulo de fusión de resultados recurre al diagnóstico probabilístico entregado por el modelo de Machine Learning (Random Forest), indicando en ambos casos el nivel de confianza asociado. Finalmente, el módulo de explicación —apoyado en el LLM— traduce el resultado técnico a una respuesta en lenguaje natural para el usuario.

Esta arquitectura híbrida busca combinar las fortalezas de cada paradigma: la explicabilidad y precisión del enfoque simbólico para los casos conocidos, la capacidad de generalización del Machine Learning para casos no cubiertos por las reglas, y la accesibilidad conversacional que aporta la IA generativa, sin sacrificar la trazabilidad del diagnóstico final.

Capítulo 8. Resultados

8.1 Sistema experto y base de conocimiento

El sistema experto implementado en Prolog respondió correctamente a las cinco consultas de prueba definidas (Anexo A), infiriendo el diagnóstico esperado en cada caso a partir de los hechos declarados. Las 15 reglas del sistema experto (Tabla 2) cubren los escenarios de falla más frecuentes reportados en la literatura de soporte técnico de redes (capa física, direccionamiento IP, DNS, autenticación y saturación de ancho de banda).

8.2 Ontología y grafo de conocimiento

La ontología desarrollada en Protégé (Anexo B) cuenta con 5 clases, 5 relaciones de tipo Object Property y 10 individuos, exportada exitosamente en formato OWL/XML y verificada sintácticamente. El grafo de conocimiento derivado (Figura 2) permitió identificar visualmente que el router_principal constituye un punto de falla común a tres incidentes distintos, información valiosa para priorizar la atención de incidentes por impacto.

8.3 Modelos de Machine Learning

La Tabla 5 resume el desempeño del Árbol de Decisión y el Random Forest (sesión 12) sobre el 20% de datos de prueba (120 casos) del dataset de 600 registros.

Modelo	Accuracy	Precision (macro)	Recall (macro)	F1-Score (macro)
Árbol de Decisión	0.800	0.721	0.631	0.657
Random Forest	0.917	0.923	0.835	0.859

La Tabla 7 resume los resultados del modelo no supervisado (sesión 13) y de la red neuronal artificial (sesión 14), evaluados sobre el mismo dataset y la misma partición de prueba que los modelos anteriores.

Modelo	Métrica principal	Valor	Observación
K-Means (no supervisado)	Silhouette Score	0.277	Separación moderada entre clústeres.
K-Means (no supervisado)	Adjusted Rand Index	0.184	Coincidencia baja-moderada con el diagnóstico real.
Red Neuronal (MLP)	Accuracy	0.725	Inferior al Random Forest en este dataset tabular.

Red Neuronal (MLP)	F1-Score (macro)	0.499	Afectado por las clases minoritarias del dataset.
--------------------	------------------	-------	---

La Tabla 9 compara de forma consolidada los cuatro modelos evaluados, permitiendo concluir que el Random Forest es el modelo recomendado para la integración con el sistema experto (Capítulo 7).

Modelo	Tipo	Accuracy	F1-Score (macro)	Interpretabilidad
Árbol de Decisión	Supervisado	0.800	0.657	Alta
Random Forest	Supervisado	0.917	0.859	Media
K-Means	No supervisado	N/A	N/A (ARI=0.184)	Media
Red Neuronal (MLP)	Supervisado	0.725	0.499	Baja

8.4 Evidencias

Las evidencias completas de ejecución (código fuente, salidas de consola, gráficas de las curvas de aprendizaje y matrices de confusión) se encuentran documentadas en los notebooks de Google Colab adjuntos como Anexos E y F, y fueron ejecutadas de extremo a extremo sin errores como parte de la validación de este informe.

Capítulo 9. Discusión

9.1 ¿Qué funcionó?

El enfoque híbrido resultó apropiado para el dominio elegido: las reglas del sistema experto cubren de forma explicable y precisa los escenarios de falla más comunes, mientras que el Random Forest demostró ser un complemento efectivo, alcanzando un desempeño superior al 90% de exactitud sin requerir el ajuste manual de reglas adicionales. La construcción de un dataset sintético alineado con las reglas del sistema experto permitió validar de forma consistente los distintos modelos de Machine Learning bajo las mismas condiciones.

9.2 ¿Qué no funcionó?

La red neuronal artificial (MLP) no superó al Random Forest, lo cual —lejos de ser un resultado negativo— confirmó empíricamente lo señalado en la literatura (Géron, 2022) sobre las limitaciones de las redes neuronales en conjuntos de datos tabulares de tamaño reducido. De igual forma, el modelo no supervisado (K-Means) mostró una correspondencia solo moderada con las categorías reales de diagnóstico, evidenciando que la sola similitud estadística entre síntomas no es suficiente para replicar el conocimiento experto codificado en las reglas.

9.3 ¿Qué aprendieron?

El equipo consolidó la comprensión de que la Inteligencia Artificial simbólica (reglas, lógica de predicados, ontologías) y la Inteligencia Artificial basada en datos (Machine Learning, redes neuronales) no son enfoques competidores sino complementarios, cada uno con fortalezas distintas: explicabilidad y precisión determinística en el primer caso, capacidad de generalización a partir de la experiencia en el segundo. También se comprendió en la práctica por qué la selección del modelo debe considerar el tamaño y la naturaleza del dataset disponible, y no únicamente la sofisticación del algoritmo.

9.4 ¿Qué mejorarían?

Con más tiempo, el equipo ampliaría el dataset con casos reales (anonimizados) de soporte técnico, lo que permitiría evaluar si la red neuronal mejora su desempeño con mayor volumen de datos, e implementaría y probaría de forma completa el prototipo del asistente conversacional basado en LLM diseñado conceptualmente en las sesiones 16 a 18.

Capítulo 10. Conclusiones

A continuación se presentan dos conclusiones por cada integrante del equipo de trabajo.

[Nombre completo del integrante 1]

1. [Conclusión 1 del integrante 1 — completar antes de la entrega final.]
2. [Conclusión 2 del integrante 1 — completar antes de la entrega final.]

[Nombre completo del integrante 2]

1. [Conclusión 1 del integrante 2 — completar antes de la entrega final.]
2. [Conclusión 2 del integrante 2 — completar antes de la entrega final.]

[Nombre completo del integrante 3]

1. [Conclusión 1 del integrante 3 — completar antes de la entrega final.]
2. [Conclusión 2 del integrante 3 — completar antes de la entrega final.]

Capítulo 11. Trabajo Futuro

11.1 Mejoras propuestas

- Reemplazar el dataset sintético por datos reales (anonimizados) de tickets de soporte técnico, para validar y reentrenar los modelos de Machine Learning bajo condiciones reales.
- Implementar un mecanismo de resolución de conflictos y priorización de reglas en el motor de inferencia, para los casos en que más de una regla del sistema experto sea aplicable simultáneamente.
- Incorporar un ciclo de reentrenamiento periódico (MLOps) que actualice el modelo de Machine Learning a medida que se acumulan nuevos casos etiquetados por los técnicos.

11.2 Nuevas funcionalidades

- Implementar de forma funcional el asistente conversacional basado en LLM diseñado en las sesiones 16 a 18, incluyendo la integración real con la API de un modelo de lenguaje.
- Desarrollar un panel de control (dashboard) que visualice el grafo de conocimiento de forma interactiva y permita a los administradores de red identificar puntos críticos de la infraestructura.
- Incorporar autenticación y control de acceso para que el sistema pueda usarse de forma segura en un entorno multiempresa.

11.3 Líneas futuras de investigación

- Aplicación de Deep Learning (redes LSTM o Temporal Convolutional Networks) sobre series temporales de tráfico de red para la detección predictiva de anomalías, antes de que se conviertan en una falla total (sesión 15).
- Evaluación de técnicas de aprendizaje semi-supervisado que combinen el pequeño volumen de casos etiquetados por expertos con el gran volumen de telemetría de red no etiquetada disponible en un entorno real.
- Estudio de técnicas de razonamiento neuro-simbólico que integren de forma más profunda el motor de reglas y los modelos de Machine Learning, más allá de la estrategia de fusión secuencial propuesta en el Capítulo 7.

Referencias Bibliográficas

- Bishop, C. M. (2006). Pattern recognition and machine learning. Springer.
- Bratko, I. (2012). Prolog programming for artificial intelligence (4th ed.). Addison-Wesley.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Clocksin, W. F., & Mellish, C. S. (2003). Programming in Prolog (5th ed.). Springer.
- Congreso de la República de Colombia. (2012). Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. *Diario Oficial* No. 48.587.
- Floridi, L., & Cowls, J. (2019). A unified framework of five principles for AI in society. *Harvard Data Science Review*, 1(1). <https://doi.org/10.1162/99608f92.8cd550d1>
- Géron, A. (2022). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.
- Giarratano, J., & Riley, G. (2005). Expert systems: Principles and programming (4th ed.). Course Technology.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
- Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220. <https://doi.org/10.1006/knac.1993.1008>
- Hogan, A., Blomqvist, E., Cochez, M., D'Amato, C., Melo, G. de, Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., Ngomo, A. C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. *ACM Computing Surveys*, 54(4), 1–37. <https://doi.org/10.1145/3447772>
- Jackson, P. (1999). Introduction to expert systems (3rd ed.). Addison-Wesley.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
<https://doi.org/10.1038/nature14539>
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
- Mitchell, T. M. (1997). Machine learning. McGraw-Hill.
- Noy, N. F., & McGuinness, D. L. (2001). Ontology development 101: A guide to creating your first ontology. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.
- OpenAI. (2023). GPT-4 technical report. arXiv. <https://arxiv.org/abs/2303.08774>

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408. <https://doi.org/10.1037/h0042519>
- Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
- Sowa, J. F. (2000). *Knowledge representation: Logical, philosophical, and computational foundations*. Brooks/Cole.
- UNESCO. (2021). *Recommendation on the ethics of artificial intelligence*. UNESCO Publishing.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv*. <https://arxiv.org/abs/2302.11382>

Anexos

Anexo A. Código fuente y base de conocimiento en Prolog

Archivo: diagnostico_red.pl — contiene los 10 hechos, 5 reglas y 5 consultas de ejemplo del sistema experto en lógica de predicados (sesión 8).

Anexo B. Ontología (.owl)

Archivo: ontologia_diagnostico_red.owl — ontología en formato OWL/XML con 5 clases, 5 relaciones y 10 individuos, compatible con Protégé (sesión 10).

Anexo C. Grafo de conocimiento

Incluido en el Capítulo 6 (Figura 2) del presente informe; generado a partir de los mismos hechos de la base de conocimiento en Prolog.

Anexo D. Dataset

Archivo: dataset_diagnostico_red.csv — 600 casos sintéticos de soporte técnico, 8 variables de entrada y 1 variable objetivo (diagnostico), usados en las sesiones 11 a 14.

Anexo E. Notebook de Google Colab — Modelo supervisado

Archivo: modelo_supervisado_diagnostico_red.ipynb — entrenamiento y evaluación del Árbol de Decisión y el Random Forest (sesión 12).

Anexo F. Notebook de Google Colab — Modelo no supervisado y red neuronal

Archivo: modelo_no_supervisado_red_neuronal.ipynb — clustering K-Means (sesión 13) y red neuronal MLP (sesión 14), ejecutado de extremo a extremo.

Anexo G. Capturas del sistema

[Adjuntar capturas de pantalla de la ejecución de las consultas Prolog, de Protégé y de los notebooks de Google Colab antes de la entrega final.]

Anexo H. Manual de usuario

[Elaborar un manual breve dirigido al usuario final, describiendo cómo reportar un síntoma y cómo interpretar el diagnóstico entregado por el sistema.]

Anexo I. Manual técnico

[Elaborar un manual técnico dirigido a otros desarrolladores, describiendo cómo instalar SWI-Prolog, cargar la base de conocimiento, abrir la ontología en Protégé y ejecutar los notebooks de Google Colab.]

Anexo J. Repositorio GitHub

[Incluir el enlace al repositorio GitHub del equipo con el código fuente completo del proyecto.]

Anexo K. Video demostrativo

[Incluir el enlace al video demostrativo del funcionamiento del sistema, elaborado por el equipo.]